



MAESTRÍA EN INTELIGENCIA ARTIFICIAL

Proyecto Integrador en Inteligencia Artificial (MIAR0545)

Reporte técnico

Diagnóstico de sobreajuste y subajuste mediante curvas de aprendizaje

Clasificación de tráfico de red cifrado sobre el dataset CESNET-TLS-Year22

Autor: Jairo Wladimir Jhayya Perlaza

Docente: Gladys María Villegas Rugel

Actividad: Semana 3. Diagnóstico de Overfitting/Underfitting

Guayaquil, octubre de 2026

1. Resumen ejecutivo

Se diagnosticó el comportamiento de los cuatro modelos del proyecto (Random Forest, XGBoost, una red convolucional 1D y un Transformer ligero), que clasifican tráfico TLS cifrado en 23 categorías de servicio a partir de metadatos de flujo. Se implementó un sistema de tracking que registra la pérdida y el F1 macro de entrenamiento y validación en cada época o ronda, con MLflow como registro de experimentos; se generaron curvas de aprendizaje por época, por tamaño del dataset y por hiperparámetro, y el diagnóstico se automatizó con métricas cuantitativas y reglas explícitas.

Hallazgo principal: el problema dominante fue el **subajuste** (3 de 4 modelos). Diagnósticos iniciales: Random Forest: subajuste; XGBoost: mixto; CNN 1D: subajuste; Transformer: subajuste. Ningún modelo presentó un comportamiento óptimo en su configuración inicial, lo que indica que todos tenían margen de mejora.

Resultados en datos nunca vistos (prueba de octubre de 2022, F1 macro): Random Forest: 0,742 → 0,838 (+9,7 puntos); XGBoost: 0,863 → 0,861 (-0,2 puntos); CNN 1D: 0,612 → 0,746 (+13,4 puntos); Transformer: 0,645 → 0,757 (+11,2 puntos).

Lección metodológica: una brecha pequeña entre entrenamiento y validación no es un objetivo en sí misma, porque puede indicar un modelo igualmente deficiente en ambos conjuntos. Por eso el diagnóstico combina la brecha con el nivel de desempeño y la tendencia de las curvas.

2. Metodología

Datos. Muestra del 2 % de CESNET-TLS-Year22: entrenamiento y validación con septiembre de 2022 (división 80/20 estratificada) y prueba con octubre. Para acotar el cómputo, el entrenamiento usó una submuestra estratificada de 200 000 flujos; la validación y la prueba se usaron completas. Antes y después se comparan con la misma muestra.

Métrica principal. F1 macro, porque el desbalance entre clases es de 145:1 y la exactitud resultaría engañosa. Es el score de la curva B de la actividad.

Tracking. Módulo `src/diagnostico/tracking.py`: (a) *RegistroMetricas*, que guarda cada paso en CSV/JSON y en MLflow (SQLite local); (b) un *callback* personalizado de XGBoost (*TrainingCallback*) que registra la pérdida en cada ronda y el F1 cada 10 rondas; (c) un bucle de PyTorch con registro manual por época; y (d) *learning_curve* y *validation_curve* de scikit-learn para Random Forest, con validación cruzada estratificada de 3 pliegues.

Sobre TensorFlow/Keras. El proyecto no usa Keras: sus redes están implementadas en PyTorch. El requisito de *callbacks* personalizados se cubrió con el mismo patrón en las dos librerías que sí usa: el *callback* de XGBoost y el registro por época del bucle de PyTorch.

Protocolo. El diagnóstico usó solo entrenamiento y validación; la prueba de octubre se usó una única vez, en la comparación final, para evitar el sobreajuste al conjunto de validación.

Métrica	Definición	Señal (umbral)
Brecha final	F1 de entrenamiento – F1 de validación	> 0,05: sobreajuste
Pendientes de pérdida	Pendiente normalizada en el último 30 % del entrenamiento	Validación sube y entrenamiento baja: sobreajuste
Nivel de F1 de entrenamiento	F1 alcanzado en entrenamiento	< 0,80 con brecha pequeña: subajuste
Mejora final	Cambio del F1 de validación en el tramo final, con el mejor punto al final	> 0,005: aún aprendía

Con señales de ambos tipos, el diagnóstico es *mixto*; sin ninguna, *óptimo*. Las reglas se verificaron con pruebas automáticas sobre curvas de forma conocida.

3. Resultados del diagnóstico

Modelo	Diagnóstico	F1 ent.	F1 val.	Brecha
Random Forest	Subajuste	0,795	0,747	0,049
XGBoost	Mixto	0,983	0,878	0,105
CNN 1D	Subajuste	0,622	0,617	0,005
Transformer	Subajuste	0,664	0,657	0,007

Ningún modelo resultó *óptimo* en su configuración inicial. No es una omisión del análisis: es el resultado, y justifica las estrategias de mejora de la sección 5.

4. Análisis de las curvas de aprendizaje

Las frases de esta sección se generan a partir de las métricas de cada curva.

CNN 1D. Antes de la estrategia, en 15 épocas, el mejor F1 de validación (0,617) se alcanzó en la época 15; al final, las curvas seguían subiendo, con una brecha de 0,005, y la pérdida de validación descendía en el tramo final. Después de la estrategia, en 40 épocas, el mejor F1 de validación (0,758) se alcanzó en la época 38; al final, las curvas seguían subiendo, con una brecha de 0,017, y la pérdida de validación descendía en el tramo final.

Transformer. Antes de la estrategia, en 15 épocas, el mejor F1 de validación (0,657) se alcanzó en la época 15; al final, las curvas seguían subiendo, con una brecha de 0,007, y la pérdida de validación descendía en el tramo final. Después de la estrategia, en 25 épocas, el mejor F1 de validación (0,769) se alcanzó en la época 25; al final, las curvas seguían subiendo, con una brecha de 0,021, y la pérdida de validación descendía en el tramo final.

XGBoost. En el tramo final, la pérdida de entrenamiento sigue bajando mientras la de validación se aplana; la brecha final de F1 es 0,105 y el mejor punto de validación está en la ronda 391 de 400.

Random Forest. La curva por tamaño muestra que el F1 de validación pasa de 0,475 con 6 666 flujos a 0,674 con 66 666, y que en el último tramo sigue creciendo: más datos todavía ayudan. En la curva de min_samples_leaf, la mejor validación (0,818) se obtiene con 1, y la estrategia eligió 1. En la curva de max_depth, la mejor validación (0,673) se obtiene con 20, y la estrategia eligió 20.

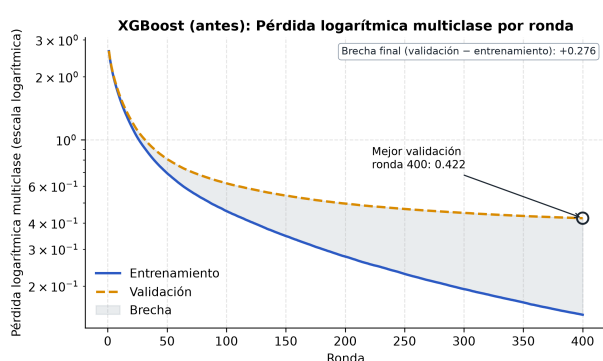


Figura 1. XGBoost (antes): pérdida por ronda (curva A).

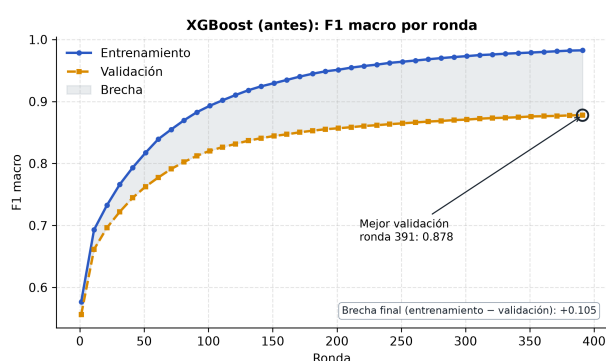


Figura 2. XGBoost (antes): F1 macro por ronda (curva B).

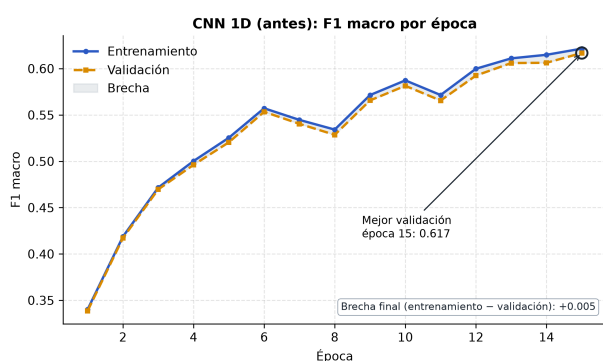


Figura 3. CNN 1D antes de la estrategia (curva B).

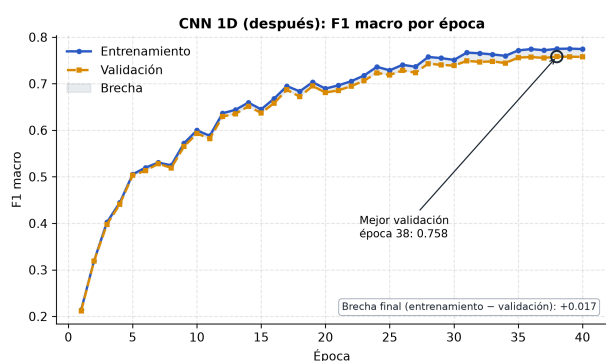


Figura 4. CNN 1D después de la estrategia (curva B).

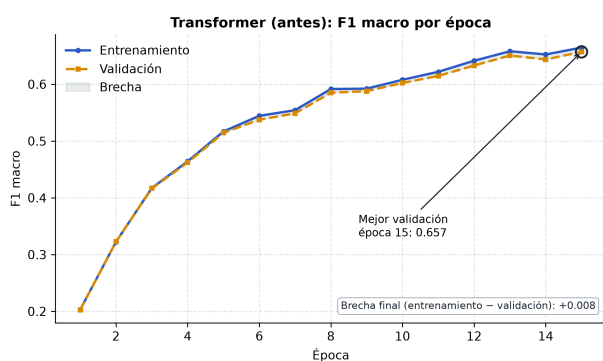


Figura 5. Transformer antes de la estrategia (curva B).

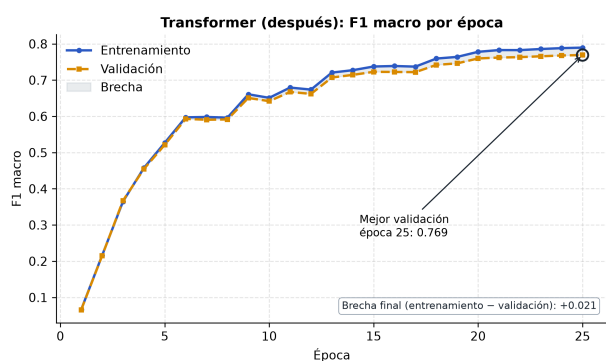


Figura 6. Transformer después de la estrategia (curva B).

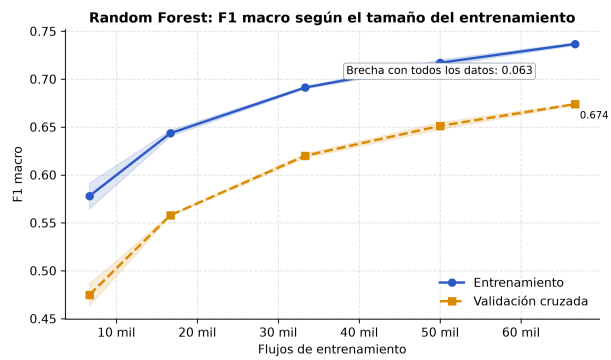


Figura 7. Random Forest: curva por tamaño del dataset (C).

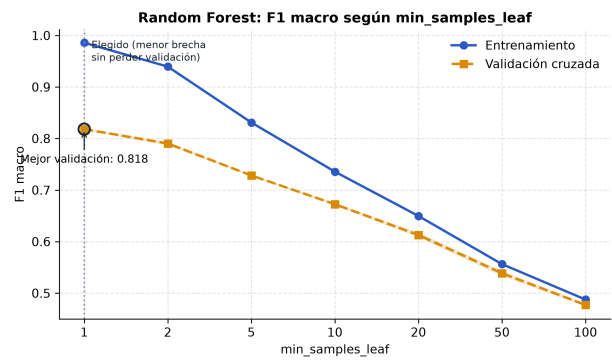


Figura 8. Random Forest: curva por min_samples_leaf (D).

5. Estrategias implementadas y resultados

Modelo	Estrategia	F1 prueba antes	F1 prueba después	Brecha antes → después
Random Forest	Ajuste de complejidad guiado por las curvas D	0,742	0,838	0,049 → 0,120
XGBoost	Regularización: submuestreo de filas y columnas 0,8, L2 = 5, min_child_weight = 5	0,863	0,861	0,105 → 0,089
CNN 1D	Más épocas, tasa de aprendizaje OneCycle y parada temprana	0,612	0,746	0,005 → 0,017
Transformer	Más épocas, tasa de aprendizaje OneCycle y parada temprana	0,645	0,757	0,007 → 0,021

Antes y después de las estrategias de mejora (validación de septiembre)

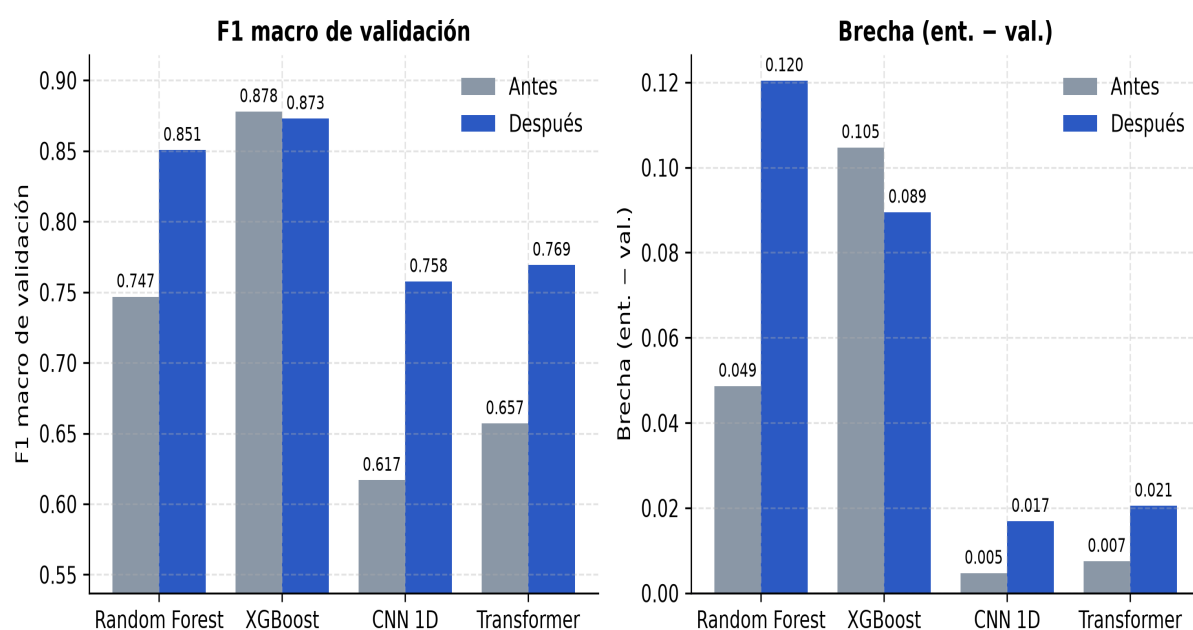


Figura 9. Comparación antes/después (validación de septiembre).

En Random Forest la brecha pasó de 0,049 a 0,120 y el F1 de prueba de 0,742 a 0,838: una brecha mayor convivió con una mejor generalización. El ajuste de complejidad se eligió con las curvas de validación, no con la prueba.

6. Conclusiones y recomendaciones futuras

1. El diagnóstico sistemático cambió la lectura del proyecto: el problema dominante fue el subajuste (3 de 4 modelos), no el sobreajuste.
2. La brecha de generalización debe interpretarse junto con el nivel de desempeño.
3. CNN 1D y Transformer todavía mejoraban al final del entrenamiento mejorado: se recomienda entrenar hasta que actúe la parada temprana y explorar mayor capacidad.
4. Repetir el análisis con los 640 000 flujos de entrenamiento y actualizar la comparación del proyecto y su aplicación web, donde este diagnóstico ya se presenta de forma interactiva.

7. Referencias técnicas

- Hynek, K., Luxemburk, J., Pešek, J., Čejka, T., y Šiška, P. (2024). CESNET-TLS-Year22: A year-spanning TLS network traffic dataset from backbone lines. *Scientific Data*, 11, 1156.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Chen, T., y Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of KDD '16*, 785-794.
- Smith, L. N., y Topin, N. (2019). Super-convergence: Very fast training of neural networks using large learning rates. *Proc. SPIE 11006*.
- Prechelt, L. (1998). Early stopping: But when? En *Neural Networks: Tricks of the Trade* (pp. 55-69). Springer.
- Goodfellow, I., Bengio, Y., y Courville, A. (2016). *Deep Learning*, caps. 5 y 7. MIT Press.
- Zaharia, M. et al. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), 39-45.
- Código, figuras, notebook y resultados: <https://github.com/jairowjp/cesnet-tls-classification>
· Aplicación web: <https://trafico-cifrado-uees.streamlit.app> (página Diagnóstico)